Lecture 22 - Data Provenance

- **Goals**:
  - understand what it means to formally track *data dependencies* and *transformation dependencies* for query results;
  - survey the major provenance models and their trade-offs in expressiveness, complexity, and supported query classes.
- Provenance answers the question: *why is a tuple in (or absent from) a query result?*
- Provenance taxonomy:
  - one axis: dependency type
    - **Data dependency**: which input tuples contributed to a result tuple?
    - **Transformation dependency**: *how* were those inputs combined by the query?
  - another axis: granularity
    - for data: relations, tuples, values
    - for transformations: queries, operators
  - each one is individually insufficient
    - e.g., data dependence insufficient for reasoning about effects of removal - without knowing the transformation
- Applications:
  - debugging, auditing, security
  - understanding the process, explanations
  - understanding outliers
  - speeding up incremental maintenance
  - reasoning about changes to inputs -> impact on outputs: what-if analysis
- Key tension in reasoning about provenance:
  - more expressive provenance models capture richer information...
  - but are harder to compute and may only work for restricted query classes.
- We may change up the order a bit to build up to the "capstone" provenance semirings approach

Provenance Setup and Properties

- Can't just "invert" queries - not invertible - need something more nuanced.
- For most of the following, will focus on RA$^+$, aka monotone queries, supporting $\sigma, \pi, \rho, U, \times$.
- Example: $\pi(R \bowtie S) = \pi(\{o_1, o_2, o_3\} \bowtie \{i_1\}) \to t$ - basically any one of the tuples o_i joined with i_i leads to t
- **Sufficiency**
  - Provenance of t should be *enough* to reproduce t.
  - **Definition**: D' ⊆ D is *sufficient* for t given Q if t ∈ Q(D').
  - For **monotone** queries, any superset of a sufficient subset is also sufficient
  - Proof: monotonicity means D' ⊆ D'' ⇒ Q(D') ⊆ Q(D''), so if t ∈ Q(D') then t ∈ Q(D'').
  - Sufficiency alone is too weak — it lets us include irrelevant tuples
- **Necessity**
  - Tries to exclude irrelevant inputs
  - **Definition**: t' ∈ D is *necessary* for t ∈ Q(D) if ∀D' ⊆ D: t ∈ Q(D') → t' ∈ D'
  - Works fine when derivations are "conjunctive"
  - Doesn't work for disjunctive case, e.g., $\pi_A(R)$ where a given A value in output has multiple tuples in R with same value of A, like o_1, o_2, o_3
- **Minimality** addresses this issue without requiring necessity:
  - **Definition**: D' is *minimal* if no proper subset of D' is sufficient.
  - Downside: for disjunctive case, could be multiple minimal subsets.
    - Which one is the provenance?
- **Summary**

    - **Sufficiency**: provenance is *enough* to reproduce the result
    - **Necessity**: excludes irrelevant inputs; fails in disjunctive contexts
    - **Minimality**: no redundant inputs; handles disjunction but yields multiple candidates
- **Syntax Independence**: Provenance model P is syntax independent if Q ≡ Q' ⇒ P(P,Q,D,t) = P(P,Q',D,t) for all D, t.
    - i.e., semantically equivalent queries should have the same provenance
    - Buneman et al. (2001) called this "*invariance under query rewriting*"
    - sufficiency, necessity, and causality (much later) are all syntax independent: they treat queries as black boxes.
    - **Caveat for transformation provenance**:
        - if provenance is used for *debugging*, it should reflect the user's specific query syntax, not just its semantics.
        - Here, syntax independence is problematic because we can't isolate to specific operations if the query format itself is malleable
- **Computability**: Provenance P is computable if Q(D) can be reconstructed from P(P,Q,D)
  - i.e., the provenance alone (without knowing Q) is sufficient to recompute the result.
    - Strictly stronger than sufficiency: provenance must encode both *which* inputs to use and *how* to combine them.
    - Why-provenance (next), necessity, and causality (much later) only capture data dependencies and do **not** have computability.
    - Provenance polynomials (later) do.

Why Provenance

- Why Provenance from Buneman et al. 2001 and then Cheney 2009: based on **sufficiency** and **minimality**
- **Definition (Witness)**: D' ⊆ D is a witness for t ∈ Q(D) if t ∈ Q(D').
- **Set of Witnesses**: Wit(Q,D,t) = {D' | D' ⊆ D ∧ t ∈ Q(D')}
- **Minimal Witnesses**: MWit(Q,D,t) = witnesses that contain no proper subset that is also a witness.
- Both *syntax independent* (defined declaratively via sufficiency/minimality).
- Buneman et al. also give a **syntax-driven operational definition** via recursive rules over RA+:
    - Base case (relation R): Why(R,D,t) = {{t}} if t ∈ R, else ∅
        - singleton tuple witness
    - Selection: Why(σ_ω(Q),D,t) = Why(Q,D,t) if t ⊨ ω, else ∅
        - go inside selections if tuple satisfies predicate
    - Projection: Why(π_A(Q),D,t) = $\cup_{u:u.A=t}$ Why(Q,D,u)
        - when going inside projections, include provenance for each tuple that projects down to t
    - Join: Why(Q1 ⋈ Q2,D,t) = {D' ∪ D'' | D' ∈ Why(Q1,D,t(Q1)) ∧ D'' ∈ Why(Q2,D,t(Q2))}
        - include all witness combos from each side of the join; t(Qi) restricts t's schema
    - Union: Why(Q1 ∪ Q2,D,t) = Why(Q1,D,t) ∪ Why(Q2,D,t)
        - include witnesses from each side of a union
- Every element of Why(Q,D,t) is a witness, and Why contains all minimal witnesses.
    - So Why(Q,D,t) = MWit(Q,D,t) after removing non-minimal elements.
- Minimal witnesses (why-provenance): $\{o_1, i_1\}, \{o_2, i_1\}, \{o_3, i_1\}$
- One limitation:
    - ignores the *multiplicity* of uses of an input tuple.
    - If a self-join uses the same tuple twice, both uses are conflated in the witness set.

Lineage-based Provenance

- From Cui and Widom, 2000
- Tracks data dependencies **per input relation** provenance is a list (R1', ..., Rn') where Ri' ⊆ Ri.
- **Declarative definition** for an operator Op with inputs R1,...,Rn: lineage of t is ⟨R1',...,Rn'⟩ such that:
    1. Op(R1',...,Rn') = {t} (strict single-output sufficiency)

2. ∀i ∀t' ∈ Ri': Op(R1',...,{t'},...,Rn') ≠ ∅ (every included tuple is individually relevant)
3. Maximal among all lists fulfilling (1) and (2).
    - ensures that it includes all relevant tuples

- **Lineage of a multi-operator query** is defined via **transitivity**:
    - if t_in is in the lineage of some intermediate tuple t_mid, and t_mid is in the lineage of the final result t, then t_in is in the lineage of t.
- **Operational definition**
    - Lin(σ_ω(R), D, t) = ⟨{t}⟩ if if t ⊨ ω, else ∅
    - Lin(π_A(R), D, t) = ⟨$\sigma_{A=t.A}$(R)⟩
    - Lin(R1 ⋈ R2, D, t) = ⟨{t(R1)}, {t(R2)}⟩
    - Lin(R1 ∪ R2, D, t) = ⟨$\sigma_{R1.Att=t}$(R1), $\sigma_{R2.Att=t}$(R2)⟩
    - Plus rules for difference, aggregation, e.g., Lin(R1 − R2, D, t) = ⟨{t}, R2⟩ (all of R2 is included as provenance — conservative)
- Unlike why provenance, **lineage is NOT syntax independent.**
    - Q1 := R vs. Q2 := $\pi_{A,B}$(R ⋈ $\rho_{B\to C}$(R))
    - Let R(A, B) = {(1, 2), (1, 3)}
    - Lin(Q1, D, (1, 2)) = ⟨ {(1, 2)} ⟩
    - Lin(Q2, D, (1, 2)) = ⟨ {(1, 2), (1, 2)}, {(1, 2), (1, 3)} ⟩ = ⟨ {(1, 2)}, {(1, 2), (1, 3)} ⟩
    - equivalent, but lineage of result tuple (1,2) from Q2 includes both (1,2) and (1,3), while Q1's lineage is just {(1,2)}.
- A bit of funkiness around the fact that the provenance mirrors the relation sequence - but can be transformed to be a subset of D lossily

Provenance Semirings

- From Green, Karvounarakis, Tannen (PODS 2007)
- The most powerful model.
- Encodes *both* data and transformation dependencies.
- Subsumes variants of why-provenance, lineage (plus set semantics, bag semantics, and incomplete databases) as special cases.

K-Relations

- Notion of K**-relations**, which is a function that annotates tuples with values from a given domain K
    - Formed as mathematical object called a **commutative semiring**
    - $(K, \oplus_K, \otimes_K, 0_K, 1_K)$

$$k_1 \oplus_{\mathcal{K}} k_2 = k_2 \oplus_{\mathcal{K}} k_1 \qquad \textbf{(commutativity of addition)}$$
$$k_1 \otimes_{\mathcal{K}} k_2 = k_2 \otimes_{\mathcal{K}} k_1 \qquad \textbf{(commutativity of multiplication)}$$
$$(k_1 \oplus_{\mathcal{K}} k_2) \oplus_{\mathcal{K}} k_3 = k_1 \oplus_{\mathcal{K}} (k_2 \oplus_{\mathcal{K}} k_3) \qquad \textbf{(associativity of addition)}$$
$$(k_1 \otimes_{\mathcal{K}} k_2) \otimes_{\mathcal{K}} k_3 = k_1 \otimes_{\mathcal{K}} (k_2 \otimes_{\mathcal{K}} k_3) \qquad \textbf{(associativity of multiplication)}$$
$$k_1 \oplus_{\mathcal{K}} \mathbb{0}_{\mathcal{K}} = k_1 \qquad \textbf{(neutral element of addition)}$$
$$k_1 \otimes_{\mathcal{K}} \mathbb{1}_{\mathcal{K}} = k_1 \qquad \textbf{(neutral element of multiplication)}$$
$$k_1 \otimes_{\mathcal{K}} \mathbb{0}_{\mathcal{K}} = \mathbb{0}_{\mathcal{K}} \qquad \textbf{(annihilation by zero)}$$
$$k_1 \otimes_{\mathcal{K}} (k_2 \oplus_{\mathcal{K}} k_3) = (k_1 \otimes_{\mathcal{K}} k_2) \oplus_{\mathcal{K}} (k_1 \otimes_{\mathcal{K}} k_3) \qquad \textbf{(multiplication distributes over addition)}$$

-
  - Each tuple that is absent implicitly gets value 0_K.
- Warmup:
  - **N** = (naturals, +, ×, 0, 1): encodes bag semantics — annotation = multiplicity
    - Note that here, we would map every non-existence tuple in the original relation as a 0
  - **B** = ({F,T}, ∨, ∧, F, T): encodes set semantics — annotation = existence
  - **W** = (2^W, ∪, ∩, ∅, W): encodes incomplete databases — annotation = set of possible worlds in which tuple exists
  - **Probability semiring** $([0,1], +, \times, 0, 1)$**:**
    - Assign each input tuple t its marginal probability `Pr(t)`.
    - Output annotation is the probability of the output tuple, assuming tuple independence.
    - Gives the *tuple-independent probabilistic database* semantics of Dalvi and Suciu.
- Operational definition for generating new tuple mappings for relational operations, independent of semiring definition
  - Rename: ($\rho_{A \to B}$(R))(t) = R(t(B→A))
  - Selection: (σ_ω(R))(t) = R(t) $\otimes_K$ ω(t) where ω(t) returns 1_K if t satisfies ω, else 0_K
  - Projection: (π_U(R))(t) = $\Sigma_{t':t'[U]=t}$ R(t') (semiring sum = addition)
  - Natural join: (R1 ⋈ R2)(t) = R1(t(R1)) $\otimes_K$ R2(t(R2)) (semiring product = multiplication)
  - Union: (R1 ∪ R2)(t) = R1(t) $\oplus_K$ R2(t)
- One-to-one correspondence between standard relational algebra equivalences
  - i.e., commutativity/associativity of ∪ and ×, distributivity and semiring laws.
- Addition is used for *disjunctive* use of inputs (union, projection); multiplication for *conjunctive* use (join, selection).
- Homomorphisms
  - Essentially: if K1 is more informative than K2 (there exists h: K1 → K2), then the K1 result of any query is sufficient to compute the K2 result.

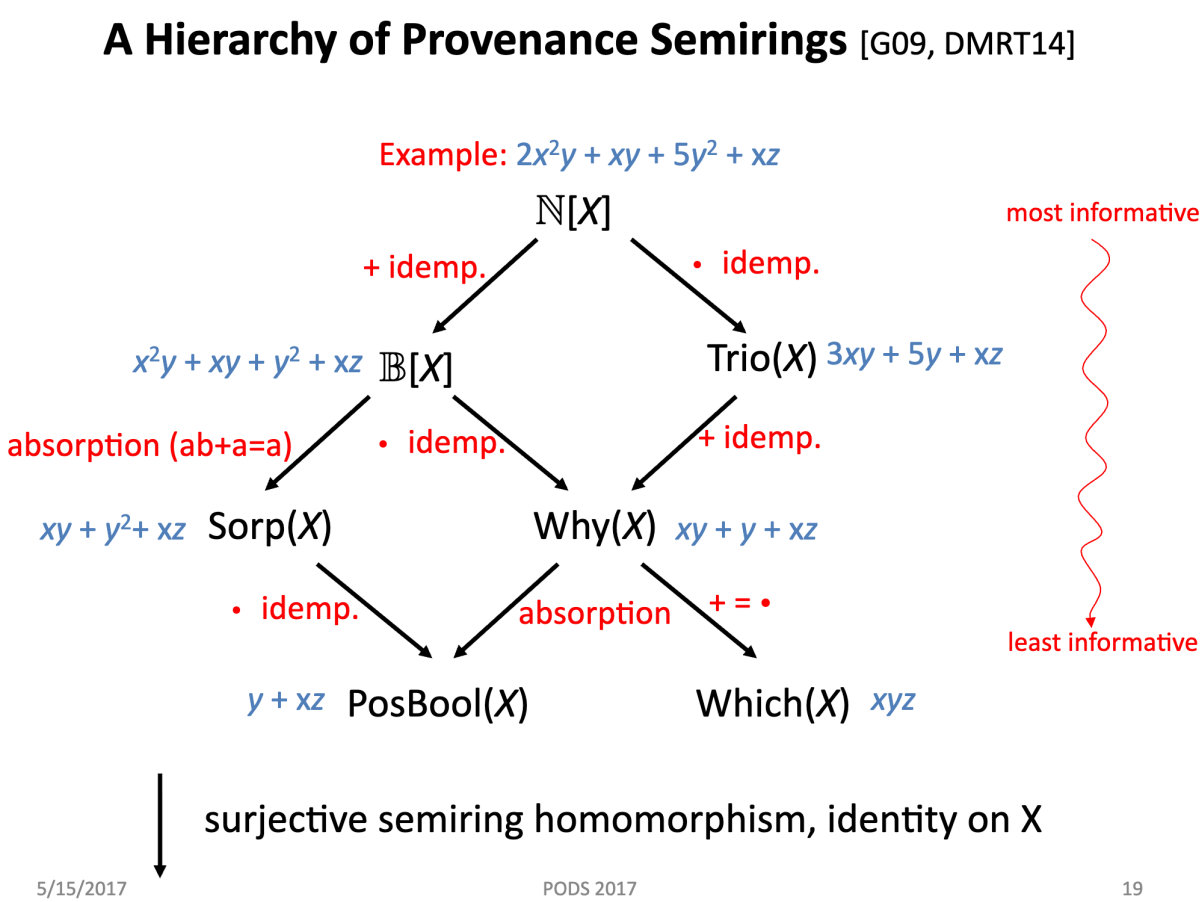
Provenance Polynomials aka How provenance

- Special semiring where we annotate, every tuple with a distinct variable (x_i for Customers, y_j for Orders, ...).
- The provenance of a result tuple is a polynomial in these variables with natural number coefficients and exponents.
  - **+** encodes alternative derivations (union, projection = disjunctive)
  - **×** encodes joint use of inputs (join = conjunctive)
- Original example $(o_1 + o_2 + o_3).i_1$
- Property: provenance polynomial has a homomorphism to any other semiring - that is, they encode enough information to reconstruct the result for any semiring
  - e.g., $(o_1 + o_2 + o_3).i_1$
  - set semantics: (T ∨ T ∨ T) ∧ T = T

- bag semantics: (1 + 1 + 1) . 1 = 3

Other Provenance Semirings

- PosBool(X) = (PosBool(X), ∨, ∧, ⊥, ⊤): elements are positive boolean formulas over a set of variables X.
  - Uses boolean algebra
  - Fulfills set-semantic-based laws, e.g., k1 ∨ k1 = k1. For
  - also: x1 + (x1 · x2) and x1 are considered to be the same in this semiring.
  - Essentially a "minimal" why semantics
- Why(X) = (2^{2^X}, ∪, ⊎, ∅, {∅})
  - Sets of tuples with union as addition and pair-wise union (⊎) as multiplication.
  - Addition and multiplication in this semiring are idempotent
  - This semiring encodes why-provenance.
- Which(X) = (2^X ∪ {⊥}, ∪+, ∪×, ⊥, ∅):
  - The elements of this semiring are sets of variables (tuple identifiers) and a special element ⊥ which denotes that a tuple does not exist.
  - This semiring corresponds to the lineage provenance model if we ignore the order of leaves in the algebra tree of the query encoded by the lineage model.
- Best illustrated in this figure from Val Tannen's talk



Other Models

Intervention-based Causality

- Introduced by Halpern and Pearl; adapted to databases by Meliou et al. (2010).
- **Counterfactual causality**: remove a candidate cause from the input and see if the result disappears.
- **Definition (Counterfactual Cause)**: t_cause ∈ D is a counterfactual cause for t ∈ Q(D) if: t ∉ Q(D \ {t_cause})
- For monotone queries, counterfactual causality ≡ necessity.
- Just like our o_1, o_2, o_3 example, counterfactual causality fails when there are alternative derivations
  - if three tuples each independently suffice, removing one does not eliminate the result.
- **Actual causality** handles alternatives via *contingencies*:
- **Definition (Actual Cause)**: t_cause is an actual cause for t ∈ Q(D) if ∃Γ ⊆ D \ {t_cause} such that:
  - t ∈ Q(D \ Γ) (Γ doesn't remove t by itself)
  - t ∉ Q(D \ Γ \ {t_cause}) (removing t_cause on top of Γ does remove t)
  - here, Γ is called the *contingency*.
- Intuition: Γ is an intervention that "disables" all alternative derivations, after which t_cause becomes counterfactually necessary.
- Every counterfactual cause is an actual cause with Γ = ∅.
- In our example:
  - o1 is an actual cause with contingency {o3, o2} (disable the other orders)
  - o3 is an actual cause with contingency {o1, o2}, etc.
  - i1 is a counterfactual cause with Γ = ∅

Responsibility

- Actual causality identifies *which* inputs are causes, but not *how much* each contributes.
- **Definition (Responsibility)**: ρ(t*cause) = 1 / (1 + min*{Γ ∈ Contingencies(t_cause)} |Γ|)
- Range: (0, 1]. A counterfactual cause (min contingency size = 0) has responsibility 1.
- Example:
  - ρ(i1) = 1/(1+0) = 1 (counterfactual, strictly necessary)
  - ρ(o1) = ρ(o2) = ρ(o3) = 1/(1+2) = 1/3 (one alternative exists)
- Responsibility extends to **aggregation**:
  - for queries like `SELECT sum(Price) FROM Items`, removal of a tuple doesn't eliminate the result tuple but changes the value.
  - Responsibility is defined as the tuple's fractional contribution to the aggregate (Roy & Suciu, 2014)

Where-Provenance (Buneman et al., 2001)

- Tracks **value copying** rather than tuple sufficiency. Provenance is defined at the level of *cells* (relation, tuple, attribute) rather than tuples.
- Where-provenance of a cell (Q, t, A) is the set of input cells whose values were (directly or indirectly through equality constraints) copied to produce t.A.
- Operational rules (Cheney et al., 2009):
  - Base: Where(R,D,t,A) = {(R,t,A)}
  - Selection: pass through
  - Projection: pass through
  - Rename: follow the rename
  - Union: union of the two sub-provenances (per attribute)
  - Join: if A is in only one schema, take that side; if shared, take union of both sides
- **Syntax dependent** — redundant self-joins can add spurious cells.
- More fine-grained than Why and How provenance, but only applies when values are precisely copied

Non-monotone Queries

- Monotone models fail for non-monotone queries because the non-existence of inputs can determine query results
- Example: `R - S` returns (1) from R={1,2}, S={2,3}.
  - Tuple (1) is in the result because R(1) exists *and* S(1) does not exist.
  - No amount of reasoning about existing tuples captures the role of the missing tuple S(1).
- Challenge: the space of missing tuples can be infinite
- For similar reasons as datalog, provenance for aggregation (also not monotone) is hard to succinctly capture